

Evaluation Framework for Text-to-SQL Agent Systems

Task 1 — Part 2

ClassicModels Database | Week 3 Assignment

1. Introduction

A Text-to-SQL agent is a system that accepts a natural language question from a user and converts it into an executable SQL query. The agent then runs the query against a database and returns a meaningful answer.

Building such a system is only half the challenge. The other half is knowing whether it actually works correctly.

This document defines a structured evaluation framework for measuring the performance of a Text-to-SQL agent. The framework covers nine evaluation dimensions, each with a clear definition, measurement method, example, and target threshold.

The 50 ground truth queries prepared in Part 1 of this task serve as the benchmark dataset. Every evaluation metric defined here will be applied when testing the agent in Tasks 3 and 4.

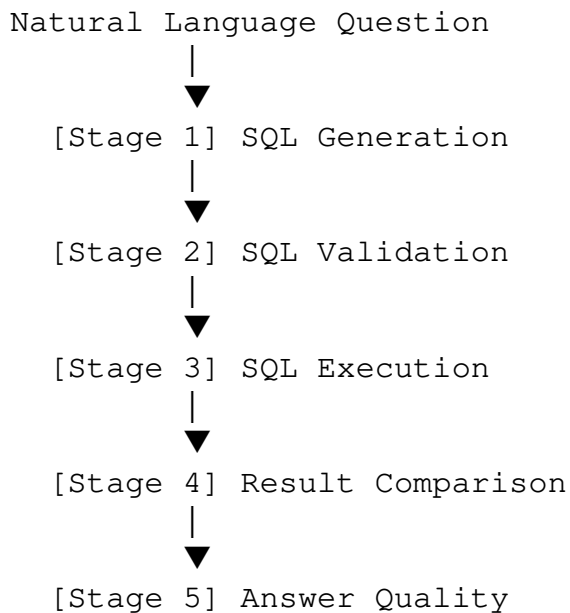
2. Why Evaluation Is Hard for Text-to-SQL

Evaluating a Text-to-SQL system is more complex than evaluating a typical software function because:

- **There is no single correct SQL query.** Multiple valid SQL queries can return the same correct result. The agent's SQL may look different from the ground truth but still be correct.
- **SQL can execute successfully but return wrong results.** A query that runs without errors can still produce incorrect output due to wrong filters, wrong joins, or wrong columns.
- **Natural language is ambiguous.** The same question can be interpreted in multiple valid ways, making it hard to define a single ground truth.
- **Failure can happen at multiple stages.** The agent can fail during SQL generation, during execution, or when formatting the final answer.

This means evaluation must happen at every stage of the pipeline, not just at the final output.

3. Evaluation Pipeline



Each stage can be evaluated independently. A failure at any stage should be recorded separately so we understand exactly where the agent breaks down.

4. Evaluation Dimensions

Dimension 1: SQL Execution Success Rate

What it measures: Whether the generated SQL query executes without throwing a database error.

Why it matters: This is the most fundamental requirement. If the SQL cannot even run, no result is returned to the user. Syntax errors, missing table names, wrong column names, and invalid operations all cause execution failure.

How to measure: Run all 50 benchmark queries through the agent. Count how many execute without error.

Execution Success Rate = (Queries that executed / Total queries) × 100

Example:

- Agent generates: `SELECT * FROM customer;`
- Error: table "customer" does not exist (correct table is "customers")
- Status: FAILED — execution error

Target threshold: $\geq 90\%$

What causes failures:

- Wrong table name (customers vs customer)
 - Wrong column name due to case sensitivity (productName vs productname)
 - Syntax errors (missing FROM clause, extra semicolons)
 - Invalid SQL operations
-

Dimension 2: Result Correctness

What it measures: Whether the query returns the correct data — not just whether it runs, but whether the answer is actually right.

Why it matters: A query can execute perfectly and still return wrong results. For example, a query counting customers in the USA might accidentally count all customers if the WHERE clause is missing. The query ran, but the answer is wrong.

How to measure: Compare the output of the agent's query against the ground truth output:

- For aggregate queries (COUNT, SUM, AVG): compare exact values
- For row-returning queries: compare row count and sample rows
- For SELECT with filters: verify the filter was applied correctly

Result Correctness Rate = (Queries with correct output / Total executed

Example:

- Question: "Total number of customers"
- Ground truth result: 122
- Agent result: 122 ✓ Correct
- Agent result: 36 ✗ Wrong (only counted USA customers)

Target threshold: $\geq 85\%$

Dimension 3: Correct Table Selection

What it measures: Whether the agent identifies and uses the correct database tables to answer the question.

Why it matters: Selecting the wrong table is a fundamental structural mistake. If the agent queries the wrong table, no amount of column or filter adjustment will produce the correct result.

How to measure: Extract the table names referenced in the generated SQL and compare them against the expected tables from the ground truth.

ClassicModels table reference:

- Customer questions → customers table
- Order questions → orders, orderdetails tables
- Product questions → products, productlines tables
- Employee questions → employees, offices tables
- Payment questions → payments table

Example:

- Question: "Get payments with customer names"
- Expected tables: payments, customers
- Agent used: payments, customers ✓
- Agent used: orders, customers ✗ Wrong table

Table Selection Accuracy = (Queries with correct tables / Total queries

Target threshold: $\geq 90\%$

Dimension 4: Correct Column Selection

What it measures: Whether the agent selects the right columns to answer the question.

Why it matters: Even with the correct table, selecting the wrong column produces misleading or incorrect results. For example, confusing buyPrice with MSRP would return vendor cost instead of retail price.

How to measure: Parse the SELECT clause of the generated SQL and compare column names against the ground truth.

Example:

- Question: "Get product names and prices"
- Expected columns: productName, buyPrice
- Agent selected: productName, buyPrice ✓
- Agent selected: productName, MSRP ✗ Wrong price column

Column Selection Accuracy = (Queries with correct columns / Total queries)

Target threshold: $\geq 85\%$

Dimension 5: Join Correctness

What it measures: Whether multi-table queries use the correct join conditions and join types.

Why it matters: An incorrect join condition produces either a cartesian product (every row matched with every row) or missing rows. Both silently return wrong data without throwing an error.

How to measure: For JOIN queries (Q21–Q30 in the benchmark), verify:

- The correct foreign key relationship is used in the ON clause
- The join type is appropriate (INNER JOIN vs LEFT JOIN)

Key relationships in ClassicModels:

orders.customerNumber	→ customers.customerNumber
employees.officeCode	→ offices.officeCode
orderdetails.productCode	→ products.productCode
products.productLine	→ productlines.productLine
customers.salesRepEmployeeNumber	→ employees.employeeNumber
employees.reportsTo	→ employees.employeeNumber (self-join)

Example:

- Question: "Get employees and their manager"
- Correct: LEFT JOIN employees m ON e.reportsTo = m.employeeNumber
- Wrong: JOIN employees m ON e.employeeNumber = m.employeeNumber (this joins employee to themselves, not to their manager)

Target threshold: $\geq 80\%$

Dimension 6: Error Handling and Retry Success Rate

What it measures: When the agent generates SQL that fails to execute, how often does it successfully recover on the first retry.

Why it matters: A production-grade agent should not simply return an error message to the user when SQL fails. It should detect the failure, analyze the error, attempt to fix the SQL, and retry once.

How to measure:

$$\text{Retry Success Rate} = (\text{Failed queries fixed on retry} / \text{Total failed queries})$$

Expected retry behavior:

1. First attempt fails with an execution error
2. Agent receives the error message
3. Agent analyzes the error and generates corrected SQL
4. Second attempt executes successfully
5. Result is returned to the user

Example:

- First attempt: `SELECT productname FROM products` Error: column "productname" does not exist
- Retry: `SELECT "productName" FROM products` ✓ Fixed

Target threshold: $\geq 70\%$ of failures resolved on first retry

Important rule: Maximum one retry is allowed. If the retry also fails, the agent should return a clear error message to the user rather than attempting further retries.

Dimension 7: Natural Language Answer Quality

What it measures: Whether the final response returned to the user is clear, accurate, and directly answers the original question in plain language.

Why it matters: The user asked a question in plain English. They should receive a plain English answer — not raw SQL, not a JSON dump, not a database error message.

How to measure (qualitative assessment): Rate each response on three criteria:

- Accuracy: Does it use the correct numbers from the query result?
- Clarity: Is it written in plain, understandable language?
- Completeness: Does it fully answer what was asked?

Examples:

Question: "How many customers are in the USA?"

✓ Good answer: "There are 36 customers located in the USA."

✗ Poor answer: "[{'count': 36}]"

✗ Poor answer: "SELECT COUNT(*) FROM customers WHERE country = 'USA' → 36"

Target: All successful queries should produce a clear natural language answer.

Dimension 8: Query Execution Latency

What it measures: The total time taken from receiving a natural language question to returning the final answer to the user.

Why it matters: A system that takes 30 seconds to answer a simple question is not useful in practice, even if it is technically correct. Latency becomes especially important when the agent makes multiple LLM calls (decomposition + generation + retry).

How to measure: Record timestamps at the start and end of each query pipeline run.

Latency = End timestamp - Start timestamp (in milliseconds)

Acceptable thresholds:

Query Type	Target Latency
Simple SELECT	< 500ms
JOIN queries	< 1000ms
Aggregation queries	< 1500ms
Retry (failed + fixed)	< 3000ms

What contributes to latency:

- LLM API call for SQL generation
 - PostgreSQL query execution time
 - LLM API call for retry (if needed)
 - LLM API call for natural language answer formatting
-

Dimension 9: Robustness to Ambiguous Questions

What it measures: How well the agent handles questions that are vague, incomplete, or open to multiple valid interpretations.

Why it matters: Real users do not always ask precise questions. A robust agent should handle ambiguity gracefully rather than producing wrong results silently or crashing.

Categories of ambiguous questions:

Type	Example	Issue
Missing filter	"Show me top customers"	Top by what metric?
Vague time range	"List recent orders"	What counts as recent?
Unclear threshold	"Get expensive products"	What price is expensive?
Multiple interpretations	"Count orders"	All orders or per customer?

Expected behavior for ambiguous questions:

- Option A: Ask for clarification before generating SQL
- Option B: Make a reasonable default assumption and state it explicitly

Example of Option B: Question: "Show top customers" Agent response: "Assuming top means highest total payment amount, here are the top 5 customers by total payments..."

What NOT to do:

- Silently pick one interpretation without telling the user
- Return an error without explanation
- Crash or return empty results

5. Evaluation Summary Table

This table will be populated during Task 3 and Task 4 testing:

#	Question	SQL Generated	Executed	Result Correct	Tables OK	Retry Needed	Final Status
1	List all products	SELECT * FROM products	✓	✓	✓	No	✓ Success
21	Get orders with customer names	SELECT ... JOIN ...	✓	✓	✓	No	✓ Success
31	Count customers per country	SELECT country, COUNT(*) ...	✓	✓	✓	No	✓ Success
...	...	...	...	...	...	...	...

6. Metrics Target Dashboard

Metric	Formula	Target
SQL Execution Success Rate	$\text{Executed} / \text{Total} \times 100$	$\geq 90\%$
Result Correctness Rate	$\text{Correct} / \text{Executed} \times 100$	$\geq 85\%$
Table Selection Accuracy	$\text{Correct tables} / \text{Total} \times 100$	$\geq 90\%$
Column Selection Accuracy	$\text{Correct columns} / \text{Total} \times 100$	$\geq 85\%$
Join Correctness	$\text{Correct joins} / \text{Join queries} \times 100$	$\geq 80\%$
Retry Success Rate	$\text{Fixed on retry} / \text{Failed} \times 100$	$\geq 70\%$
Avg Query Latency	Total time / Queries	$< 1000\text{ms}$

7. How This Framework Connects to Tasks 3 and 4

Task 3 — Text-to-SQL Pipeline: The pipeline will be run against all 50 benchmark questions. Dimensions 1, 2, 3, 4, 5, and 8 will be measured automatically by comparing generated SQL output against the ground truth established in Part 1.

Task 4 — Agentic System: The retry and self-correction mechanism will be measured using

Dimension 6. Natural language answer quality (Dimension 7) and ambiguity handling (Dimension 9) will also be assessed in the full agentic system.

8. Conclusion

Evaluating a Text-to-SQL agent requires looking beyond simple execution success. A truly good agent must select the right tables, the right columns, use correct join conditions, handle failures gracefully, respond in clear natural language, and do all of this within an acceptable time frame.

The nine dimensions defined in this framework provide a complete picture of agent performance. The 50 ground truth queries from Part 1 serve as the reference dataset for all evaluation throughout this module.